为什么需要内存对齐?

charonwchen 冯诺没有依曼 2025年08月30日 22:30 广东

为什么需要内存对齐?

刚学 C 语言的时候，就听说 `struct` 中的成员和 `struct` 本身会进行对齐（alignment）。但是不清楚具体的对齐规则是什么？为什么需要对齐？感觉非常玄乎！现在心血来潮准备花费一番力气搞搞清楚，以夯实自己的计算机基础。

struct 的 alignment 规则

在介绍 `struct` 的 alignment 之前，首先需要解释什么是自然对齐（`natural alignment`）。
每种数据类型都有由硬件决定的自然对齐需求，自然对齐由平台ABI + 编译器实现。需要注意的是 `alignment≠size`。

`struct` 的 `alignment` 规则如下:

- `struct` 本身按照成员对齐值中的最大值进行对齐
- 每个成员按照各自的自然对齐值进行对齐；为满足下一个成员的对齐，编译器会在成员之间插入 `padding`

例子如下:

```
// sizeof(GeneralStruct1) = 8, alignof(GeneralStruct1) = max(alignof(c),alignof(s),alignof(i)) = 4
// 以偏移量从0开始，进行 GeneralStruct1 的布局推导
// c 在 0（占 1）
// short 的对齐值为 2 ，因此需要在 s 前 padding 1 字节，使 s 起始偏移为 2，s 在 2..3（占 2）
```

```
// 现在偏移 = 4，i 需要 4 对齐，已经满足，所以 s 与 i 之间无需 padding，i 在 4..7（占 4）
// 现在偏移 = 8，且 8 是结构体对齐 4 的倍数，无需 padding
struct GeneralStruct1{
  char c;// one byte
  short s;// two bytes
  int i;// four bytes
};

// sizeof(GeneralStruct2) = 12,, alignof(GeneralStruct2) = max(alignof(c),alignof(i),alignof(s)) = 4
// 以偏移量从0开始，进行 GeneralStruct2 的布局推导
// c 在 0（占 1）
// 在 i 前 padding 3 字节，使 i 起始偏移为 4（满足 4 对齐），i 在 4..7（占 4）
// 现在偏移 = 8，s 需要 2 对齐，已满足，所以 i 与 s 之间无 padding，s 在 8..9（占 2）
// 为使 GeneralStruct2 满足 4 对齐，在末尾补 2 字节
struct GeneralStruct2{
  char c;// one byte
  int i;// four bytes
  short s;// two bytes
};

// 可以使用宏 offsetof 来查看成员的偏移，以此来验证推导是否正确，用法如：offsetof(GeneralStruct1, s)
```

内存对齐的原因

内存对齐的原因涉及到硬件、编译器，后文将一一介绍

硬件层面

硬件层面的内存对齐原因涉及到内存、缓存和处理器层面

内存层面

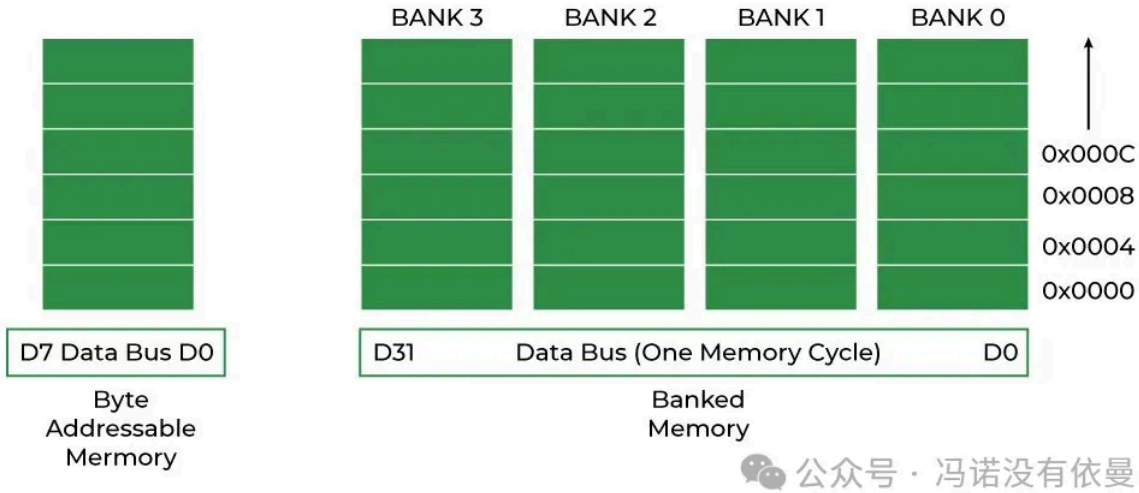

图1 不同memory中的bank排列

内存是按顺序排布、按字节寻址的。如果 memory 被排列成单字节宽度的单 `bank` （如上图 1 左侧所示），那么处理器在寻址一个 `int` 的时候需要 4 次内存周期，才能拿到这个 `int` 值。为了在一次内存周期内拿到 `int` 值，需要将 4 个单字节宽度的 `bank` 水平排列成如上图 1 右侧所示即可。

在 memory 排列如上图 1 右侧所示时，假设 `bank0..3` 分别有地址 `x` 、 `x+1` 、 `x+2` 和 `x+3` （ `x` 是4的倍数），此时假设一个 `int` 分配在地址 `x` 上，处理器将在一个内存周期内读取这个 `int` ；假设为 `int` 分配的地址是 `y` （不是 4 的倍数），那么它将跨越两个 bank 行，如下图 2 所示，将需要两个内存读取周期来读取。

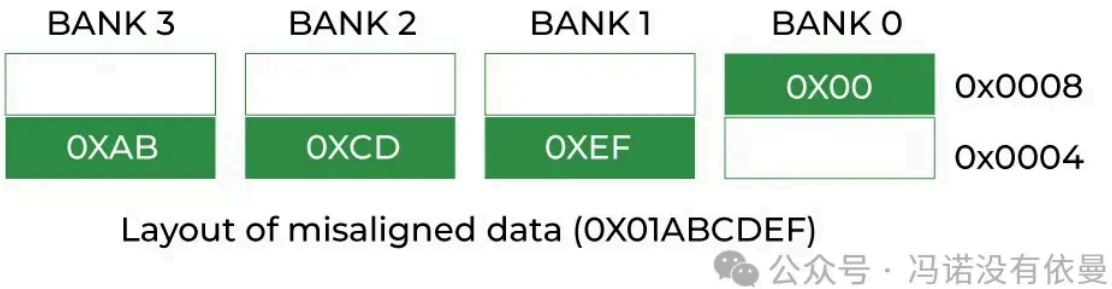

图2 int分布在两个bank行

cache 层面

数据不对齐，可能会造成其跨越多个 `cache line`，在使用这个数据的时候，就会使用两个 `cache line` （一般长度是 64B），造成性能下降。

数据不对齐时，在多核并行访问下可能会造成 `cache` 同步的消耗增加，此时需要同步两个 `cache line` 。

处理器层面

```
// 在内存中紧密存储 sizeof(PackedStruct) = 7
struct __attribute__((packed)) PackedStruct {
    char c;  // one byte
    int i;   // four bytes
    short s; // two bytes
};
```

当结构体不内存对齐时，如上面的结构体 `PackedStruct` ，其在内存中的排列如下图 3 所示。

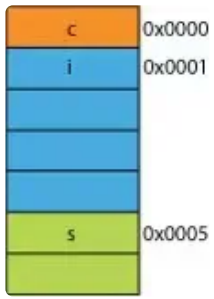

图3 PackedStruct在内存中的排列图

某些处理器对内存访问有严格对齐要求，如果数据的内存不是对齐的，会触发硬件异常。即使处理器支持不对齐的数据访问，对于像上述结构体 `PackedStruct` 中的成员 `int i` 的读取，它也将先访问地址 `0x0000` （以图3 这样的内存布局为例），将其读取到 `register a` ，然后往左偏移一个字节，然后访问地址 `0x0001` ，将其读取到 `register b` ，然后将其往左偏移三个字节，最后将前述两者进行 `OR` 操作，才能得到这个 `int` 值。

编译器层面

编译器在布局结构体时，会依据目标平台的 `ABI（应用二进制接口）` 规定和处理器架构的对齐约定，为每个成员安排合适的地址偏移，并插入填充字节（padding）以满足对齐要求。

编译器可以通过 `__attribute__((packed))` （这个属性一般用于对象需要网络传输，以节省带宽；或者用于某些数据库场景，以节省内存占用）等手段来改变编译器对齐规则，但是这会影响性能，前面硬件层面已经进行了分析。

cpp 中的 over-aligned

在 cpp 中可以通过 `alignof(std::max_align_t)` 来得到所有标量类型（算数类型和指针类型）所需对齐的最大值。对于类型 `T`，如果满足 `alignof(T) ≤ alignof(std::max_align_t)` 那么称 `T` 为基本对齐（fundamental alignment）类型[6]。如果满足 `alignof(T) > alignof(std::max_align_t)` ，那么称 `T` 为 `over-aligned` 类型。

如果一个类 S，它的任何一个成员都不是 `over-aligned` 类型，并且也没有用 `alignment specifier` （例如 `alignas` ） 指定 `extended alignment` （使 `alignment` 大于 `alignof(std::max_align_t)` ），那么 S 就是 `fundamental alignment` 类型。如果一个类 T，它的任何一个成员是 `over-aligned` 类型，或者用 `alignment specifier` （例如 `alignas` ）指定了 `extended alignment` ，那么 `T` 就是 `over-aligned` 类型。

从 cpp11 开始，可以通过 `alignas` 指定类型或者对象的对齐要求。需要注意的是：

- 对齐必须是 `2` 的幂次
- 使用 `alignas` 指定对齐要求不能比没有 `alignas` 时更弱，这是不合法的。

例子如下：

```
struct alignas(7) T {}; // error: requested alignment is not a power of 2

struct alignas(8) S {};
struct alignas(1) U { S s; }; // error: requested alignment is less than minimum alignment of 8 for type 'U'
```

为什么需要 `over-aligned` 类型?

`over-aligned` 的使用场景包括

- 在使用 SIMD 技术时对数据进行对齐，如 SSE 按 16B，AVX2 按 32B，AVX-512 按 64B 对齐
- 避免伪共享（false sharing），如把频繁写的队列头尾按缓存行（常 64B）对齐，让不同核的访问的数据落在不同 cache line，减少 cache line 同步开销
- 高性能 I/O 对齐约束，如 O_DIRECT 文件 I/O，需要 buffer 对齐到设备逻辑块大小（一般为 512 B）

在用 SSE 指令集进行加速时，可能需要定义如下类型：

```
// 如果 vec4 没有 alignas(16)，那么自然对齐是 4
class alignas(16) vec4
{
  float x, y, z, w;
};

auto pVectors = new vec4[1000];
```

即使指定了 `alignas(16)`，cpp17 之前不保证内存将被对齐到 `16B`，各家编译器对 `new` 遇到 `over-aligned` 类型时的做法是不一致的。当然，可以通过使用如 `posix_memalign` 这样的 posix api 来分配对齐的内存，但是这样就需要通过 `free` 来释放内存。此时只能使用裸的内存操作，无法使用智能指针来进行内存管理（因为智能指针使用 `delete` 进行内存释放，混用会导致未定义行为）。

cpp17 引入了对齐版的 `operator new/delete`（带 `std::align_val_t` 入参），并规定：当类型 `T` 是 `alignof(T) > alignof(std::max_align_t)` 时，`new T` 会自动调用对齐分配函数，`delete` 也会自动匹配到对齐释放函数。当需要 `new` 一个 `Over-Aligned` 对象T时，`operator new` 的 `std::align_val_t` 类型的入参值是 `alignof(T)`。

```
// cpp17 引入
// std::align_val_t 是一个枚举类，继承自 size_t
// 对齐版本的 operator new/delete 也是用户可以重写的
void* operator new(size_t, std::align_val_t);
void* operator new[](size_t, std::align_val_t);
void operator delete(void*, std::align_val_t);
void operator delete[](void*, std::align_val_t);
```

参考

[1] https://stackoverflow.com/questions/381244/purpose-of-memory-alignment

[2] https://stackoverflow.com/questions/63234968/width-of-bus-betwen-cpu-cache-and-cpu

[3] https://stackoverflow.com/questions/3655926/why-does-cpu-access-memory-on-a-word-boundary

[4] https://www.geeksforgeeks.org/c/structure-member-alignment-padding-and-data-packing/

[5] https://fylux.github.io/2017/07/11/Memory_Alignment/#:~:text=Let%E2%80%99s%20suppose%20that%20cache%20lines,fit%20in%20a%20cache%20line

[6] https://en.cppreference.com/w/cpp/types/max_align_t.html

[7] https://en.cppreference.com/w/cpp/memory/new/operator_new.html